

in each dimension to be equal or unequal, according to taste.) With four parameters and three cars, for example, you end up with $3 \times 3 \times 3 \times 3 = 81$ cells.

Next, choose M cells to contain the sample points by the following algorithm: Randomly choose one of the M^N cells for the first point. Now eliminate all cells that agree with this point on *any* of its parameters (that is, cross out all cells in the same row, column, etc.), leaving $(M - 1)^N$ candidates. Randomly choose one of these, eliminate new rows and columns, and continue the process until there is only one cell left, which then contains the final sample point.

The result of this construction is that *each* design parameter will have been tested in *every one* of its subranges. If the response of the system under test is dominated by *one* of the design parameters (the *main effect*), that parameter will be found with this sampling technique. On the other hand, if there are important *interaction effects* among different design parameters, then the Latin hypercube gives no particular advantage. Use with care.

There is a large field in statistics that deals with *design of experiments*. A brief pedagogical introduction is [5].

CITED REFERENCES AND FURTHER READING:

- Halton, J.H. 1960, "On the Efficiency of Certain Quasi-Random Sequences of Points in Evaluating Multi-dimensional Integrals," *Numerische Mathematik*, vol. 2, pp. 84–90.[1]
- Bratley P., and Fox, B.L. 1988, "Implementing Sobol's Quasirandom Sequence Generator," *ACM Transactions on Mathematical Software*, vol. 14, pp. 88–100.[2]
- Lambert, J.P. 1988, "Quasi-Random Sequences in Numerical Practice," in *Numerical Mathematics — Singapore 1988*, ISNM vol. 86, R.P. Agarwal, Y.M. Chow, and S.J. Wilson, eds. (Basel: Birkhäuser), pp. 273–284.
- Niederreiter, H. 1988, "Quasi-Monte Carlo Methods for Multidimensional Numerical Integration," in *Numerical Integration III*, ISNM vol. 85, H. Brass and G. Hämmerlin, eds. (Basel: Birkhäuser), pp. 157–171.
- Sobol', I.M. 1967, "On the Distribution of Points in a Cube and the Approximate Evaluation of Integrals," *USSR Computational Mathematics and Mathematical Physics*, vol. 7, no. 4, pp. 86–112.[3]
- Antonov, I.A., and Saleev, V.M. 1979, "An Economic Method of Computing I_{p_1} Sequences," *USSR Computational Mathematics and Mathematical Physics*, vol. 19, no. 1, pp. 252–256.[4]
- Dunn, O.J., and Clark, V.A. 1974, *Applied Statistics: Analysis of Variance and Regression* (New York, Wiley) [discusses Latin Square].
- Czitrom, V. 1999, "One-Factor-at-a-Time Versus Designed Experiments," *The American Statistician*, vol. 53, pp. 126–131, online at <http://www.amstat.org/publications/tas/czitrom.pdf>. [5]

7.9 Adaptive and Recursive Monte Carlo Methods

This section discusses more advanced techniques of Monte Carlo integration. As examples of the use of these techniques, we include two rather different, fairly sophisticated, multidimensional Monte Carlo codes: *vegas* [1,2], and *miser* [4]. The techniques that we discuss all fall under the general rubric of *reduction of variance* (§7.7), but are otherwise quite distinct.

7.9.1 Importance Sampling

The use of *importance sampling* was already implicit in equations (7.7.6) and (7.7.7). We now return to it in a slightly more formal way. Suppose that an integrand f can be written as the product of a function h that is almost constant times another, positive, function g . Then its integral over a multidimensional volume V is

$$\int f dV = \int (f/g) g dV = \int h g dV \quad (7.9.1)$$

In equation (7.7.7) we interpreted equation (7.9.1) as suggesting a change of variable to G , the indefinite integral of g . That made $g dV$ a perfect differential. We then proceeded to use the basic theorem of Monte Carlo integration, equation (7.7.1). A more general interpretation of equation (7.9.1) is that we can integrate f by instead sampling h — not, however, with uniform probability density dV , but rather with nonuniform density $g dV$. In this second interpretation, the first interpretation follows as the special case, where the *means* of generating the nonuniform sampling of $g dV$ is via the transformation method, using the indefinite integral G (see §7.3).

More directly, one can go back and generalize the basic theorem (7.7.1) to the case of nonuniform sampling: Suppose that points x_i are chosen within the volume V with a probability density p satisfying

$$\int p dV = 1 \quad (7.9.2)$$

The generalized fundamental theorem is that the integral of any function f is estimated, using N sample points $x_0, \dots, x_{N-1}$, by

$$I \equiv \int f dV = \int \frac{f}{p} p dV \approx \left\langle \frac{f}{p} \right\rangle \pm \sqrt{\frac{\langle f^2/p^2 \rangle - \langle f/p \rangle^2}{N}} \quad (7.9.3)$$

where angle brackets denote arithmetic means over the N points, exactly as in equation (7.7.2). As in equation (7.7.1), the “plus-or-minus” term is a one standard deviation error estimate. Notice that equation (7.7.1) is in fact the special case of equation (7.9.3), with $p = \text{constant} = 1/V$.

What is the best choice for the sampling density p ? Intuitively, we have already seen that the idea is to make $h = f/p$ as close to constant as possible. We can be more rigorous by focusing on the numerator inside the square root in equation (7.9.3), which is the variance per sample point. Both angle brackets are themselves Monte Carlo estimators of integrals, so we can write

$$S \equiv \left\langle \frac{f^2}{p^2} \right\rangle - \left\langle \frac{f}{p} \right\rangle^2 \approx \int \frac{f^2}{p^2} p dV - \left[\int \frac{f}{p} p dV \right]^2 = \int \frac{f^2}{p} dV - \left[\int f dV \right]^2 \quad (7.9.4)$$

We now find the optimal p subject to the constraint equation (7.9.2) by the functional variation

$$0 = \frac{\delta}{\delta p} \left(\int \frac{f^2}{p} dV - \left[\int f dV \right]^2 + \lambda \int p dV \right) \quad (7.9.5)$$

with λ a Lagrange multiplier. Note that the middle term does not depend on p . The variation (which comes inside the integrals) gives $0 = -f^2/p^2 + \lambda$ or

$$p = \frac{|f|}{\sqrt{\lambda}} = \frac{|f|}{\int |f| dV} \quad (7.9.6)$$

where λ has been chosen to enforce the constraint (7.9.2).

If f has one sign in the region of integration, then we get the obvious result that the optimal choice of p — if one can figure out a practical way of effecting the sampling — is that it be proportional to $|f|$. Then the variance is reduced to zero. Not so obvious, but seen

to be true, is the fact that $p \propto |f|$ is optimal even if f takes on both signs. In that case the variance per sample point (from equations 7.9.4 and 7.9.6) is

$$S = S_{\text{optimal}} = \left(\int |f| dV \right)^2 - \left(\int f dV \right)^2 \quad (7.9.7)$$

One curiosity is that one can add a constant to the integrand to make it all of one sign, since this changes the integral by a known amount, $\text{constant} \times V$. Then, the optimal choice of p always gives zero variance, that is, a perfectly accurate integral! The resolution of this seeming paradox (already mentioned at the end of §7.7) is that perfect knowledge of p in equation (7.9.6) requires perfect knowledge of $\int |f| dV$, which is tantamount to already knowing the integral you are trying to compute!

If your function f takes on a known constant value in most of the volume V , it is certainly a good idea to add a constant so as to make that value zero. Having done that, the accuracy attainable by importance sampling depends in practice not on how small equation (7.9.7) is, but rather on how small is equation (7.9.4) for an *implementable* p , likely only a crude approximation to the ideal.

7.9.2 Stratified Sampling

The idea of *stratified sampling* is quite different from importance sampling. Let us expand our notation slightly and let $\langle\langle f \rangle\rangle$ denote the true average of the function f over the volume V (namely the integral divided by V), while $\langle f \rangle$ denotes as before the simplest (uniformly sampled) Monte Carlo *estimator* of that average:

$$\langle\langle f \rangle\rangle \equiv \frac{1}{V} \int f dV \quad \langle f \rangle \equiv \frac{1}{N} \sum_i f(x_i) \quad (7.9.8)$$

The variance of the estimator, $\text{Var}(\langle f \rangle)$, which measures the square of the error of the Monte Carlo integration, is asymptotically related to the variance of the function, $\text{Var}(f) \equiv \langle\langle f^2 \rangle\rangle - \langle\langle f \rangle\rangle^2$, by the relation

$$\text{Var}(\langle f \rangle) = \frac{\text{Var}(f)}{N} \quad (7.9.9)$$

(compare equation 7.7.1).

Suppose we divide the volume V into two equal, disjoint subvolumes, denoted a and b , and sample $N/2$ points in each subvolume. Then another estimator for $\langle\langle f \rangle\rangle$, different from equation (7.9.8), which we denote $\langle f \rangle'$, is

$$\langle f \rangle' \equiv \frac{1}{2} (\langle f \rangle_a + \langle f \rangle_b) \quad (7.9.10)$$

in other words, the mean of the sample averages in the two half-regions. The variance of estimator (7.9.10) is given by

$$\begin{aligned} \text{Var}(\langle f \rangle') &= \frac{1}{4} [\text{Var}(\langle f \rangle_a) + \text{Var}(\langle f \rangle_b)] \\ &= \frac{1}{4} \left[\frac{\text{Var}_a(f)}{N/2} + \frac{\text{Var}_b(f)}{N/2} \right] \\ &= \frac{1}{2N} [\text{Var}_a(f) + \text{Var}_b(f)] \end{aligned} \quad (7.9.11)$$

Here $\text{Var}_a(f)$ denotes the variance of f in subregion a , that is, $\langle\langle f^2 \rangle\rangle_a - \langle\langle f \rangle\rangle_a^2$, and correspondingly for b .

From the definitions already given, it is not difficult to prove the relation

$$\text{Var}(f) = \frac{1}{2} [\text{Var}_a(f) + \text{Var}_b(f)] + \frac{1}{4} (\langle\langle f \rangle\rangle_a - \langle\langle f \rangle\rangle_b)^2 \quad (7.9.12)$$

(In physics, this formula for combining second moments is the “parallel axis theorem.”) Comparing equations (7.9.9), (7.9.11), and (7.9.12), one sees that the stratified (into two subvolumes) sampling gives a variance that is never larger than the simple Monte Carlo case — and smaller whenever the means of the stratified samples, $\langle\langle f \rangle\rangle_a$ and $\langle\langle f \rangle\rangle_b$, are different.

We have not yet exploited the possibility of sampling the two subvolumes with *different numbers* of points, say N_a in subregion a and $N_b \equiv N - N_a$ in subregion b . Let us do so now. Then the variance of the estimator is

$$\text{Var}(\langle f \rangle') = \frac{1}{4} \left[\frac{\text{Var}_a(f)}{N_a} + \frac{\text{Var}_b(f)}{N - N_a} \right] \quad (7.9.13)$$

which is minimized (one can easily verify) when

$$\frac{N_a}{N} = \frac{\sigma_a}{\sigma_a + \sigma_b} \quad (7.9.14)$$

Here we have adopted the shorthand notation $\sigma_a \equiv [\text{Var}_a(f)]^{1/2}$, and correspondingly for b . If N_a satisfies equation (7.9.14), then equation (7.9.13) reduces to

$$\text{Var}(\langle f \rangle') = \frac{(\sigma_a + \sigma_b)^2}{4N} \quad (7.9.15)$$

Equation (7.9.15) reduces to equation (7.9.9) if $\text{Var}(f) = \text{Var}_a(f) = \text{Var}_b(f)$, in which case stratifying the sample makes no difference.

A standard way to generalize the above result is to consider the volume V divided into more than two equal subregions. One can readily obtain the result that the optimal allocation of sample points among the regions is to have the number of points in each region j proportional to σ_j (that is, the square root of the variance of the function f in that subregion). In spaces of high dimensionality (say $d \gtrsim 4$) this is not in practice very useful, however. Dividing a volume into K segments along each dimension implies K^d subvolumes, typically much too large a number when one contemplates estimating all the corresponding σ_j 's.

7.9.3 Mixed Strategies

Importance sampling and stratified sampling seem, at first sight, inconsistent with each other. The former concentrates sample points where the magnitude of the integrand $|f|$ is largest, the latter where the variance of f is largest. How can both be right?

The answer is that (like so much else in life) it all depends on what you know and how well you know it. Importance sampling depends on already knowing some approximation to your integral, so that you are able to generate random points x_i with the desired probability density p . To the extent that your p is not ideal, you are left with an error that decreases only as $N^{-1/2}$. Things are particularly bad if your p is far from ideal in a region where the integrand f is changing rapidly, since then the sampled function $h = f/p$ will have a large variance. Importance sampling works by smoothing the values of the sampled function h and is effective only to the extent that you succeed in this.

Stratified sampling, by contrast, does not necessarily require that you know anything about f . Stratified sampling works by smoothing out the fluctuations of the *number* of points in subregions, not by smoothing the values of the points. The simplest stratified strategy, dividing V into N equal subregions and choosing one point randomly in each subregion, already gives a method whose error decreases asymptotically as N^{-1} , much faster than $N^{-1/2}$. (Note that quasi-random numbers, §7.8, are another way of smoothing fluctuations in the density of points, giving nearly as good a result as the “blind” stratification strategy.)

However, “asymptotically” is an important caveat: For example, if the integrand is negligible in all but a single subregion, then the resulting one-sample integration is all but useless. Information, even very crude, allowing importance sampling to put many points in the active subregion would be much better than blind stratified sampling.

Stratified sampling really comes into its own if you have some way of estimating the variances, so that you can put unequal numbers of points in different subregions, according

to (7.9.14) or its generalizations, *and* if you can find a way of dividing a region into a practical number of subregions (notably *not* K^d with large dimension d), while yet significantly reducing the variance of the function in each subregion compared to its variance in the full volume. Doing this requires a lot of knowledge about f , though different knowledge from what is required for importance sampling.

In practice, importance sampling and stratified sampling are not incompatible. In many, if not most, cases of interest, the integrand f is small everywhere in V except for a small fractional volume of “active regions.” In these regions the magnitude of $|f|$ and the standard deviation $\sigma = [\text{Var}(f)]^{1/2}$ are comparable in size, so both techniques will give about the same concentration of points. In more sophisticated implementations, it is also possible to “nest” the two techniques, so that, e.g., importance sampling on a crude grid is followed by stratification within each grid cell.

7.9.4 Adaptive Monte Carlo: VEGAS

The VEGAS algorithm, invented by Peter Lepage [1,2], is widely used for multidimensional integrals that occur in elementary particle physics. VEGAS is primarily based on importance sampling, but it also does some stratified sampling if the dimension d is small enough to avoid K^d explosion (specifically, if $(K/2)^d < N/2$, with N the number of sample points). The basic technique for importance sampling in VEGAS is to construct, adaptively, a multidimensional weight function g that is *separable*,

$$p \propto g(x, y, z, \dots) = g_x(x)g_y(y)g_z(z) \dots \quad (7.9.16)$$

Such a function avoids the K^d explosion in two ways: (i) It can be stored in the computer as d separate one-dimensional functions, each defined by K tabulated values, say — so that $K \times d$ replaces K^d . (ii) It can be sampled as a probability density by consecutively sampling the d one-dimensional functions to obtain coordinate vector components $(x, y, z, \dots)$.

The optimal separable weight function can be shown to be [1]

$$g_x(x) \propto \left[\int dy \int dz \dots \frac{f^2(x, y, z, \dots)}{g_y(y)g_z(z) \dots} \right]^{1/2} \quad (7.9.17)$$

(and correspondingly for $y, z, \dots$). Notice that this reduces to $g \propto |f|$ (7.9.6) in one dimension. Equation (7.9.17) immediately suggests VEGAS’ adaptive strategy: Given a set of g -functions (initially all constant, say), one samples the function f , accumulating not only the overall estimator of the integral, but also the Kd estimators (K subdivisions of the independent variable in each of d dimensions) of the right-hand side of equation (7.9.17). These then determine improved g functions for the next iteration.

When the integrand f is concentrated in one, or at most a few, regions in d -space, then the weight function g ’s quickly become large at coordinate values that are the projections of these regions onto the coordinate axes. The accuracy of the Monte Carlo integration is then enormously enhanced over what simple Monte Carlo would give.

The weakness of VEGAS is the obvious one: To the extent that the projection of the function f onto individual coordinate directions is uniform, VEGAS gives no concentration of sample points in those dimensions. The worst case for VEGAS, e.g., is an integrand that is concentrated close to a body diagonal line, e.g., one from $(0, 0, 0, \dots)$ to $(1, 1, 1, \dots)$. Since this geometry is completely nonseparable, VEGAS can give no advantage at all. More generally, VEGAS may not do well when the integrand is concentrated in one-dimensional (or higher) curved trajectories (or hypersurfaces), unless these happen to be oriented close to the coordinate directions.

The routine `vegas` that follows is essentially Lepage’s standard version, minimally modified to conform to our conventions. (We thank Lepage for permission to reproduce the program here.) For consistency with other versions of the VEGAS algorithm in circulation, we have preserved original variable names. The parameter `NDMX` is what we have called K , the maximum number of increments along each axis; `MXDIM` is the maximum value of d ; some other parameters are explained in the comments.

The vegas routine performs $m = \text{itmx}$ statistically independent evaluations of the desired integral, each with $N = \text{ncall}$ function evaluations. While statistically independent, these iterations do assist each other, since each one is used to refine the sampling grid for the next one. The results of all iterations are combined into a single best answer, and its estimated error, by the relations

$$I_{\text{best}} = \sum_{i=0}^{m-1} \frac{I_i}{\sigma_i^2} / \sum_{i=0}^{m-1} \frac{1}{\sigma_i^2} \quad \sigma_{\text{best}} = \left(\sum_{i=0}^{m-1} \frac{1}{\sigma_i^2} \right)^{-1/2} \quad (7.9.18)$$

Also returned is the quantity

$$\chi^2/m \equiv \frac{1}{m-1} \sum_{i=0}^{m-1} \frac{(I_i - I_{\text{best}})^2}{\sigma_i^2} \quad (7.9.19)$$

If this is significantly larger than 1, then the results of the iterations are statistically inconsistent, and the answers are suspect.

Here is the interface to vegas. (The full code is given in [3].)

```
void vegas(VecDoub_I &regn, Doub fxn(VecDoub_I &, const Doub), const Int init,
          const Int ncall, const Int itmx, const Int nprn, Doub &tgral, Doub &sd,
          Doub &chi2a) {
```

Performs Monte Carlo integration of a user-supplied ndim -dimensional function fxn over a rectangular volume specified by $\text{regn}[0..2*\text{ndim}-1]$, a vector consisting of ndim “lower left” coordinates of the region followed by ndim “upper right” coordinates. The integration consists of itmx iterations, each with approximately ncall calls to the function. After each iteration the grid is refined; more than 5 or 10 iterations are rarely useful. The input flag init signals whether this call is a new start or a subsequent call for additional iterations (see comments in the code). The input flag nprn (normally 0) controls the amount of diagnostic output. Returned answers are tgral (the best estimate of the integral), sd (its standard deviation), and chi2a (χ^2 per degree of freedom, an indicator of whether consistent results are being obtained). See text for further details.

The input flag init can be used to advantage. One might have a call with $\text{init}=0$, $\text{ncall}=1000$, $\text{itmx}=5$ immediately followed by a call with $\text{init}=1$, $\text{ncall}=100000$, $\text{itmx}=1$. The effect would be to develop a sampling grid over five iterations of a small number of samples, then to do a single high accuracy integration on the optimized grid.

To use vegas for the torus example discussed in §7.7 (the density integrand only, say), the function fxn would be

```
Doub torusfunc(const VecDoub &x, const Doub wgt) {
    Doub den = exp(5.*x[2]);
    if (SQR(x[2])+SQR(sqrt(SQR(x[0])+SQR(x[1]))-3.) <= 1.) return den;
    else return 0.;
}
```

and the main code would be

```
Doub tgral, sd, chi2a;
VecDoub regn(6);
regn[0] = 1.; regn[3] = 4.;
regn[1] = -3.; regn[4] = 4.;
regn[2] = -1.; regn[5] = 1.;
vegas(regn,torusfunc,0,10000,10,0,tgral,sd,chi2a);
vegas(regn,torusfunc,1,900000,1,0,tgral,sd,chi2a);
```

Note that the user-supplied integrand function, fxn , has an argument wgt in addition to the expected evaluation point $\mathbf{x}$. In most applications you ignore wgt inside the function. Occasionally, however, you may want to integrate some additional function or functions along with the principal function f . The integral of any such function g can be estimated by

$$I_g = \sum_i w_i g(\mathbf{x}) \quad (7.9.20)$$

where the w_i 's and $\mathbf{x}$'s are the arguments `wgt` and `x`, respectively. It is straightforward to accumulate this sum inside your function `fxn` and to pass the answer back to your main program via global variables. Of course, $g(\mathbf{x})$ had better resemble the principal function f to some degree, since the sampling will be optimized for f .

The full listing of `vegas` is given in a Webnote [3].

7.9.5 Recursive Stratified Sampling

The problem with stratified sampling, we have seen, is that it may not avoid the K^d explosion inherent in the obvious, Cartesian, tessellation of a d -dimensional volume. A technique called *recursive stratified sampling* [4] attempts to do this by successive bisections of a volume, not along all d dimensions, but rather along only one dimension at a time. The starting points are equations (7.9.10) and (7.9.13), applied to bisections of successively smaller subregions.

Suppose that we have a quota of N evaluations of the function f and want to evaluate $\langle f \rangle'$ in the rectangular parallelepiped region $R = (\mathbf{x}_a, \mathbf{x}_b)$. (We denote such a region by the two coordinate vectors of its diagonally opposite corners.) First, we allocate a fraction p of N toward exploring the variance of f in R : We sample pN function values uniformly in R and accumulate the sums that will give the d different pairs of variances corresponding to the d different coordinate directions along which R can be bisected. In other words, in pN samples, we estimate $\text{Var}(f)$ in each of the regions resulting from a possible bisection of R ,

$$\begin{aligned} R_{ai} &\equiv (\mathbf{x}_a, \mathbf{x}_b - \tfrac{1}{2}\mathbf{e}_i \cdot (\mathbf{x}_b - \mathbf{x}_a)\mathbf{e}_i) \\ R_{bi} &\equiv (\mathbf{x}_a + \tfrac{1}{2}\mathbf{e}_i \cdot (\mathbf{x}_b - \mathbf{x}_a)\mathbf{e}_i, \mathbf{x}_b) \end{aligned} \quad (7.9.21)$$

Here $\mathbf{e}_i$ is the unit vector in the i th coordinate direction, $i = 1, 2, \dots, d$.

Second, we inspect the variances to find the most favorable dimension i to bisect. By equation (7.9.15), we could, for example, choose that i for which the sum of the square roots of the variance estimators in regions R_{ai} and R_{bi} is minimized. (Actually, as we will explain, we do something slightly different.)

Third, we allocate the remaining $(1-p)N$ function evaluations between the regions R_{ai} and R_{bi} . If we used equation (7.9.15) to choose i , we should do this allocation according to equation (7.9.14).

We now have two parallelepipeds, each with its own allocation of function evaluations for estimating the mean of f . Our “RSS” algorithm now shows itself to be *recursive*: To evaluate the mean in each region, we go back to the sentence beginning “First,...” in the paragraph above equation (7.9.21). (Of course, when the allocation of points to a region falls below some number, we resort to simple Monte Carlo rather than continue with the recursion.)

Finally, we combine the means and also estimated variances of the two subvolumes using equation (7.9.10) and the first line of equation (7.9.11).

This completes the RSS algorithm in its simplest form. Before we describe some additional tricks under the general rubric of “implementation details,” we need to return briefly to equations (7.9.13) – (7.9.15) and derive the equations that we actually use instead of these. The right-hand side of equation (7.9.13) applies the familiar scaling law of equation (7.9.9) twice, once to a and again to b . This would be correct if the estimates $\langle f \rangle_a$ and $\langle f \rangle_b$ were each made by simple Monte Carlo, with uniformly random sample points. However, the two estimates of the mean are in fact made recursively. Thus, there is no reason to expect equation (7.9.9) to hold. Rather, we might substitute for equation (7.9.13) the relation,

$$\text{Var}(\langle f \rangle') = \frac{1}{4} \left[\frac{\text{Var}_a(f)}{N_a^\alpha} + \frac{\text{Var}_b(f)}{(N - N_a)^\alpha} \right] \quad (7.9.22)$$

where α is an unknown constant ≥ 1 (the case of equality corresponding to simple Monte Carlo). In that case, a short calculation shows that $\text{Var}(\langle f \rangle')$ is minimized when

$$\frac{N_a}{N} = \frac{\text{Var}_a(f)^{1/(1+\alpha)}}{\text{Var}_a(f)^{1/(1+\alpha)} + \text{Var}_b(f)^{1/(1+\alpha)}} \quad (7.9.23)$$

and that its minimum value is

$$\text{Var}(\langle f \rangle') \propto \left[\text{Var}_a(f)^{1/(1+\alpha)} + \text{Var}_b(f)^{1/(1+\alpha)} \right]^{1+\alpha} \quad (7.9.24)$$

Equations (7.9.22) – (7.9.24) reduce to equations (7.9.13) – (7.9.15) when $\alpha = 1$. Numerical experiments to find a self-consistent value for α find that $\alpha \approx 2$. That is, when equation (7.9.23) with $\alpha = 2$ is used recursively to allocate sample opportunities, the observed variance of the RSS algorithm goes approximately as N^{-2} , while any other value of α in equation (7.9.23) gives a poorer fall-off. (The sensitivity to α is, however, not very great; it is not known whether $\alpha = 2$ is an analytically justifiable result or only a useful heuristic.)

The principal difference between `miser`'s implementation and the algorithm as described thus far lies in how the variances on the right-hand side of equation (7.9.23) are estimated. We find empirically that it is somewhat more robust to use the square of the difference of maximum and minimum sampled function values, instead of the genuine second moment of the samples. This estimator is of course increasingly biased with increasing sample size; however, equation (7.9.23) uses it only to compare two subvolumes (a and b) having approximately equal numbers of samples. The “max minus min” estimator proves its worth when the preliminary sampling yields only a single point, or a small number of points, in active regions of the integrand. In many realistic cases, these are indicators of nearby regions of even greater importance, and it is useful to let them attract the greater sampling weight that “max minus min” provides.

A second modification embodied in the code is the introduction of a “dithering parameter,” `dith`, whose nonzero value causes subvolumes to be divided not exactly down the middle, but rather into fractions $0.5 \pm \text{dith}$, with the sign of the $\pm$ randomly chosen by a built-in random number routine. Normally `dith` can be set to zero. However, there is a large advantage in taking `dith` to be nonzero if some special symmetry of the integrand puts the active region exactly at the midpoint of the region, or at the center of some power-of-two submultiple of the region. One wants to avoid the extreme case of the active region being evenly divided into 2^d abutting corners of a d -dimensional space. A typical nonzero value of `dith`, on those occasions when it is useful, might be 0.1. Of course, when the dithering parameter is nonzero, we must take the differing sizes of the subvolumes into account; the code does this through the variable `frac1`.

One final feature in the code deserves mention. The RSS algorithm uses a single set of sample points to evaluate equation (7.9.23) in all d directions. At the bottom levels of the recursion, the number of sample points can be quite small. Although rare, it can happen that in one direction all the samples are in one half of the volume; in that case, that direction is ignored as a candidate for bifurcation. Even more rare is the possibility that all of the samples are in one half of the volume in *all* directions. In this case, a random direction is chosen. If this happens too often in your application, then you should increase `MNPT` (see line `if (jb == -1)...` in the code).

Note that `miser`, as given, returns as `ave` an estimate of the average function value $\langle f \rangle$, not the integral of f over the region. The routine `vegas`, adopting the other convention, returns as `tgral` the integral. The two conventions are of course trivially related, by equation (7.9.8), since the volume V of the rectangular region is known.

The interface to the `miser` routine is this:

```
void miser(Doub func(VecDoub_I &), VecDoub_I &regn, const Int npts,
           const Doub dith, Doub &ave, Doub &var) {
```

Monte Carlo samples a user-supplied `ndim`-dimensional function `func` in a rectangular volume specified by `regn[0..2*ndim-1]`, a vector consisting of `ndim` “lower-left” coordinates of the region followed by `ndim` “upper-right” coordinates. The function is sampled a total of `npts` times, at locations determined by the method of recursive stratified sampling. The mean value of the function in the region is returned as `ave`; an estimate of the statistical uncertainty of `ave` (square of standard deviation) is returned as `var`. The input parameter `dith` should normally be set to zero, but can be set to (e.g.) 0.1 if `func`'s active region falls on the boundary of a power-of-two subdivision of region.

Implementing code for the torus problem in §7.7 is


```

Doub torusfunc(const VecDoub &x) {
    Doub den = exp(5.*x[2]);
    if (SQR(x[2])+SQR(sqrt(SQR(x[0])+SQR(x[1]))-3.) <= 1.) return den;
    else return 0.;
}

```

and the main code is

```

Doub ave, var, tgral, sd, vol = 3.*7.*2.;
regn[0] = 1.; regn[3] = 4.;
regn[1] = -3.; regn[4] = 4.;
regn[2] = -1.; regn[5] = 1.;
miser(torusfunc,regn,1000000,0.,ave,var);
tgral = ave*vol;
sd = sqrt(var)*vol;

```

(Actually, miser is not particularly well-suited to this problem.)

The complete listing of miser is given in a Webnote [5]. The miser routine calls the short function ranpt to get a random point within a specified d -dimensional region. The version of ranpt in the Webnote makes consecutive calls to a uniform random number generator and does the obvious scaling. One can easily modify ranpt to generate its points via the quasi-random routine sobseq (§7.8). We find that miser with sobseq can be considerably more accurate than miser with uniform random deviates. Since the use of RSS and the use of quasi-random numbers are completely separable, however, we have not made the code given here dependent on sobseq. A similar remark might be made regarding importance sampling, which could in principle be combined with RSS. (One could in principle combine vegas and miser, although the programming would be intricate.)

CITED REFERENCES AND FURTHER READING:

- Hammersley, J.M. and Handscomb, D.C. 1964, *Monte Carlo Methods* (London: Methuen).
- Kalos, M.H. and Whitlock, P.A. 1986, *Monte Carlo Methods* (New York: Wiley).
- Bratley, P., Fox, B.L., and Schrage, E.L. 1983, *A Guide to Simulation*, 2nd ed. (New York: Springer).
- Lepage, G.P. 1978, "A New Algorithm for Adaptive Multidimensional Integration," *Journal of Computational Physics*, vol. 27, pp. 192–203.[1]
- Lepage, G.P. 1980, "VEGAS: An Adaptive Multidimensional Integration Program," Publication CLNS-80/447, Cornell University.[2]
- Numerical Recipes Software 2007, "Complete VEGAS Code Listing," *Numerical Recipes Webnote No. 9*, at <http://www.nr.com/webnotes?9> [3]
- Press, W.H., and Farrar, G.R. 1990, "Recursive Stratified Sampling for Multidimensional Monte Carlo Integration," *Computers in Physics*, vol. 4, pp. 190–195.[4]
- Numerical Recipes Software 2007, "Complete Miser Code Listing," *Numerical Recipes Webnote No. 10*, at <http://www.nr.com/webnotes?10> [5]